

CN Lab Manual 02

Python Data Structures

Instructor: Muhammad Nouman Hanif



Department of Computer Science, FAST-NU, Lahore, Pakistan

National University of Computer and Emerging Sciences

Fall 2025

Contents

1	Objectives	2
2	Task Distribution	2
3	Python Lists	2
3.1	Indexing & Slicing Examples	2
3.2	List Modification Examples	3
3.3	List functions	3
3.4	List comprehensions	4
4	Python Tuples	5
4.1	Tuple Functions	5
5	Python Dictionaries	5
5.1	Dictionary Modification and Formatting	6
5.2	Dictionary Functions	6
6	Exercise (40 Marks)	8
7	Submission Instructions	10

1 Objectives

After performing this lab, students shall be able to understand Python data structures which includes:

- Python lists
- Python tuples
- Python dictionaries

2 Task Distribution

The lab is designed to be completed within **170 minutes**, with the following time allocation for each section:

Topic	Time Allotment
Lab 1 Revision	20 Minutes
Python Lists	20 Minutes
Python Tuples	10 Minutes
Python Dictionaries	20 Minutes
Exercise	90 Minutes
Online Submission	10 Minutes
Total Time	170 Minutes

3 Python Lists

In Python, everything is treated as an object. Lists represent ordered sequences of values. They are **mutable**, meaning they can be modified "in place". List elements are accessed with square brackets, and Python uses zero-based indexing (the first element is at index 0).

Example 1: List Creation

```
1 # List of integers
2 primes = [2, 3, 5, 7]
3
4 # List of strings
5 planets = ['Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter', 'Saturn', 'Uranus', '
    Neptune']
6
7 # List of lists
8 hands = [
9     ['J', 'Q', 'K'],
10    ['2', '2', '2'],
11    ['6', 'A', 'K'], # A comma after the last element is optional
12 ]
13
14 # A list can contain a mix of different types
15 my_favourite_things = [32, 'CN Lab', 100.25]
```

3.1 Indexing & Slicing Examples

You can access elements from the start or end of the list using an index. Slicing lets you extract a sub-list by specifying a start and end index.

Example 2: Indexing and Slicing

```
1 planets = ['Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter', 'Saturn', 'Uranus', 'Neptune']
2
3 # Indexing
4 print(planets[0])    # 'Mercury'
5 print(planets[-1])   # 'Neptune' (last element)
6 print(planets[-2])   # 'Uranus' (second to last)
7
8 # Slicing
9 print(planets[0:3])  # ['Mercury', 'Venus', 'Earth']
10 print(planets[:3])  # Same as above
11 print(planets[3:])  # ['Mars', 'Jupiter', 'Saturn', 'Uranus', 'Neptune']
12 print(planets[1:-1])# ['Venus', 'Earth', 'Mars', 'Jupiter', 'Saturn', 'Uranus']
13 print(planets[-3:]) # ['Saturn', 'Uranus', 'Neptune']
```

3.2 List Modification Examples

Since lists are mutable, you can change their content by assigning new values to specific indices or slices.

Example 3: List Modification

```
1 planets = ['Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter', 'Saturn', 'Uranus', 'Neptune']
2
3 # Rename Mars
4 planets[3] = 'Malacandra'
5 print(planets)
6
7 # Rename the first three elements
8 planets[:3] = ['Mur', 'Vee', 'Ur']
9 print(planets)
```

3.3 List functions

Python has several useful built-in functions (like `len()`) and list methods (like `.append()`) for working with lists.

Example 4: List Functions and Methods

```

1 planets = ['Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter', 'Saturn', 'Uranus', '
  Neptune']
2 primes = [2, 3, 5, 7]
3
4 # Functions that work on lists
5 print(f"Length of planets: {len(planets)}")
6 print(f"Sorted planets: {sorted(planets)}")
7 print(f"Sum of primes: {sum(primes)}")
8 print(f"Max of primes: {max(primes)}")
9
10 # List methods
11 planets.append('Pluto') # Add an item to the end
12 print(f"After append: {planets}")
13 print(f"Popped item: {planets.pop()}") # Remove and return the last item
14 print(f"After pop: {planets}")
15
16 # Find an element's index
17 print(f"Index of Earth: {planets.index('Earth')}")
18
19 # Check for existence
20 print(f"Is Earth a planet? {'Earth' in planets}")
21
22 # Remove elements by index or slice
23 a = [-1, 1, 66.25, 333, 333, 1234.5]
24 del a[0] # Remove first element
25 print(f"List 'a' after del a[0]: {a}")
26 del a[2:4] # Remove a slice
27 print(f"List 'a' after del a[2:4]: {a}")

```

3.4 List comprehensions

List comprehensions are a unique Python feature for creating lists from other iterables in a concise way, often combining a loop and a condition into a single line.

Example 5: List Comprehension

```

1 # Without list comprehension
2 squares = []
3 for n in range(10):
4     squares.append(n**2)
5 print(f"Squares (traditional): {squares}")
6
7 # With list comprehension
8 squares_comp = [n**2 for n in range(10)]
9 print(f"Squares (comprehension): {squares_comp}")
10
11 # Filtering and transformations
12 planets = ['Mercury', 'Venus', 'Earth', 'Mars', 'Jupiter', 'Saturn', 'Uranus', '
  Neptune']
13 short_planets = [p for p in planets if len(p) < 6]
14 print(f"Short planets: {short_planets}")
15
16 loud_short_planets = [p.upper() + '!' for p in planets if len(p) < 6]
17 print(f"Loud short planets: {loud_short_planets}")
18
19 # One-line solution for counting negative numbers
20 def count_negatives(nums):
21     return sum([num < 0 for num in nums])
22
23 print(f"Negative count: {count_negatives([5, -1, -2, 0, 3])}")

```

4 Python Tuples

Tuples are almost the same as lists, with two key differences:

1. The syntax for creating them uses parentheses `()` instead of square brackets `[]`.
2. They cannot be modified (they are `**immutable**`).

Example 6: Tuples

```
1 # Tuple creation
2 t = (1, 2, 3)
3 print(t)
4
5 # Parentheses are optional
6 t_no_paren = 1, 2, 3
7 print(t == t_no_paren) # True
8
9 # Immutability example
10 try:
11     t[0] = 100
12 except TypeError as e:
13     print(f"Error: {e}")
14
15 # Classic Python swapping trick uses tuple packing/unpacking
16 a = 1
17 b = 0
18 a, b = b, a # a becomes 0, b becomes 1
19 print(f"a = {a}, b = {b}")
```

4.1 Tuple Functions

Because tuples are immutable, they have very few methods. The two most common are `.count()` and `.index()`.

Example 7: Tuple Methods

```
1 thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
2 # Count occurrences of 5
3 x = thistuple.count(5)
4 print(f"Count of 5 is: {x}")
5
6 # Find the first index of 8
7 y = thistuple.index(8)
8 print(f"Index of 8 is: {y}")
```

5 Python Dictionaries

Dictionaries are a core Python data structure for storing data in key-value pairs. They are mutable and dynamic like lists, but elements are accessed via `**keys**` instead of numerical indices. Keys must be unique and of an immutable type (e.g., string, number, or tuple).

Example 8: Dictionary Creation

```

1 # Method 1: Curly braces
2 mlb_team = {
3     'Colorado': 'Rockies',
4     'Boston': 'Red Sox',
5     'Minnesota': 'Twins',
6 }
7 print(mlb_team)
8
9 # Method 2: dict() constructor with tuples
10 mlb_team2 = dict([
11     ('Milwaukee', 'Brewers'),
12     ('Seattle', 'Mariners')
13 ])
14 print(mlb_team2)
15
16 # Method 3: dict() with keyword arguments
17 tel = dict(sape=4139, guido=4127, jack=409)
18 print(tel)
19
20 # Method 4: Dictionary comprehension
21 squares_dict = {x: x**2 for x in (2, 4, 6)}
22 print(squares_dict)
23
24 # Method 5: Building a dictionary incrementally
25 person = {}
26 person['fname'] = 'Joe'
27 person['lname'] = 'Fonebone'
28 person['age'] = 51
29 person['children'] = ['Ralph', 'Betty', 'Joey']
30 print(person)

```

5.1 Dictionary Modification and Formatting

You can easily add, update, and retrieve values from a dictionary using their keys. String formatting can also leverage dictionary keys for clean, readable code.

Example 9: Modification and Formatting

```

1 mlb_team = {'Colorado': 'Rockies', 'Boston': 'Red Sox'}
2
3 # Retrieve a value
4 print(mlb_team['Boston'])
5
6 # Add a new entry
7 mlb_team['Kansas City'] = 'Royals'
8 print(mlb_team)
9
10 # Update an entry
11 mlb_team['Boston'] = 'Bruins'
12 print(mlb_team)
13
14 # Formatting with the % operator
15 hash_map = {'count': 42, 'word': 'garfield'}
16 s = 'I want %(count)d copies of %(word)s' % hash_map
17 print(s)

```

5.2 Dictionary Functions

Dictionaries come with several useful methods for retrieving data, removing items, and merging with other dictionaries.

Example 10: Dictionary Methods

```
1 d = {'a': 10, 'b': 20, 'c': 30}
2
3 # Get a value (safer than direct access)
4 print(f"Value of 'b': {d.get('b')}")
5 print(f"Value of 'x': {d.get('x', 0)}") # With default value
6
7 # Get all items, keys, and values
8 print(f"Items: {list(d.items())}")
9 print(f"Keys: {list(d.keys())}")
10 print(f"Values: {list(d.values())}")
11
12 # Pop an item
13 popped_value = d.pop('b')
14 print(f"Popped value for 'b': {popped_value}")
15 print(f"Dict after pop: {d}")
16
17 # Update with another dictionary
18 d2 = {'a': 100, 'd': 400}
19 d.update(d2)
20 print(f"Dict after update: {d}")
21
22 # Clear the dictionary
23 d.clear()
24 print(f"Dict after clear: {d}")
```

6 Exercise (40 Marks)

Each question is worth 2 marks.

1. **Greeting Formatter:** Given a 'name' (string) and 'age' (int), return a formatted string. If age is 18 or over, return "NAME is AGE years old.". If age is under 18, return "NAME is a minor.".

Hint

Use an 'if/else' statement and f-strings for formatting.

2. **Middle Character:** Given a string 's', return the middle character. If 's' has an even length, return the two middle characters.

Hint

Check if the length of the string is odd or even using the modulus (%) operator. Use slicing to extract the character(s).

3. **Count Vowels:** Given a string 's', return the count of vowels ('a, e, i, o, u'), case-insensitive.

Hint

Iterate through the string (converted to lowercase) and check if each character is in the string "aeiou".

4. **Word Mixer:** Given two strings, 'a' and 'b', return a single string in the format 'a-b-b-a'.

Hint

Use f-strings or the '+' operator for string concatenation.

5. **Running Sum:** Given a list of numbers, return a new list where each element is the sum of all previous elements in the original list, including itself. E.g., '[1, 2, 3]' returns '[1, 3, 6]'.

Hint

Keep a 'current_sum' variable that you update as you loop through the list, appending the new sum to your result list at each step.

6. **Find Max Difference:** Given a list of numbers, find and return the difference between the largest and smallest number.

Hint

Use the built-in 'max()' and 'min()' functions.

7. **Rotate List Left:** Given a list 'nums' and an integer 'k', return a new list that is rotated to the left by 'k' positions. E.g., '[1,2,3,4,5]' with 'k=2' returns '[3,4,5,1,2]'.

Hint

Use list slicing. The new list is the part from index 'k' to the end, followed by the part from the beginning to index 'k'.

8. **Remove Even Numbers:** Given a list of integers, return a new list with all even numbers removed.

Hint

Use a list comprehension with a condition that checks if a number is odd ('num % 2 != 0').

9. **Cumulative Product:** Given a list of numbers, return the product of all the numbers in the list.

Hint

Initialize a 'product' variable to 1 and multiply it by each number in a loop.

10. **Sort by String Length:** Given a list of strings, sort it and return the new list in ascending order based on the length of each string.

Hint

Use the 'sorted()' function with a custom 'key=len' argument.

11. **Find Median:** Given a list of numbers, return the median. You will need to sort the list first. For an even-sized list, the median is the average of the two middle elements.

Hint

Sort the list. Find the middle index. If the list length is odd, the median is the element at the middle index. If even, it's the average of the two central elements.

12. **Check if Sorted:** Given a list, return 'True' if it is sorted in non-decreasing order, and 'False' otherwise.

Hint

Compare the list to a sorted version of itself: 'my_list == sorted(my_list)'.

13. **Second Largest Element:** Given a list of numbers, find and return the second largest element. Assume the list has at least two unique elements.

Hint

Convert the list to a set to remove duplicates, sort it, and then get the second-to-last element.

14. **Bubble Sort Pass:** Given a list of numbers, perform a single pass of the bubble sort algorithm (iterate through the list once, swapping adjacent elements if they are in the wrong order) and return the modified list.

Hint

Use a 'for' loop that goes up to 'len(list) - 1'. Inside, compare 'list[i]' with 'list[i+1]' and swap if needed.

15. **Invert Dictionary:** Given a dictionary, create and return a new dictionary where keys become values and values become keys. Assume all values in the input are unique and can be used as keys.

Hint

Use a dictionary comprehension: 'value: key for key, value in my_dict.items()'.

16. **Dictionary from Lists:** Given two lists, one of 'keys' and one of 'values', create and return a new dictionary mapping each key to its corresponding value.

Hint

Use the 'zip()' function combined with the 'dict()' constructor: 'dict(zip(keys, values))'.

17. **Merge Dictionaries (Value Sum):** Given two dictionaries with numeric values, merge them. If a key exists in both, the value in the new dictionary should be the sum of the values from both.

Hint

Create a copy of the first dictionary. Then, iterate through the second dictionary. If a key exists in your copy, add the values. If not, add the new key-value pair.

18. **Character Frequency:** Given a string, return a dictionary where keys are the characters in the string and values are their frequencies.

Hint

Create an empty dictionary. Loop through the string. For each character, use `dict.get(char, 0) + 1` to update its count.

19. **Filter Dictionary by Value:** Given a dictionary with numeric values and a 'threshold' number, return a new dictionary containing only the key-value pairs where the value is greater than the 'threshold'.

Hint

Use a dictionary comprehension with an 'if' condition on the value.

20. **Set Difference:** Given two lists, return a new list containing elements that are present in the first list but not in the second list.

Hint

Convert both lists to sets and then find the difference using the '-' operator.

7 Submission Instructions

Always read the submission instructions carefully.

- Rename your Jupyter notebook to your roll number and download it as a '.ipynb' file.
- To download, go to **File - Download .ipynb**. Only submit this file.
- DO NOT zip or rar your submission file.
- Submit this file on Google Classroom under the relevant assignment. Late submissions will not be accepted.